*** * * IN THE NAME OF GOD * * ***

Ali Amini & Abolfazl Najm

****

40231990 & 40231967

Shiraz University_ faculty of computer engineering

GPU Programming

GPU PROJECT

Professor: Dr.Khunjush

CUDA-Accelerated N-Body Solar System Simulation

1. Introduction and Physical Model

The objective of this project is to simulate the gravitational interactions of an N-body system representing a solar system. Calculating the gravitational pull between every pair of bodies requires an $O(N^2)$ time complexity, making it computationally expensive and an ideal candidate for parallel execution on a Graphics Processing Unit (GPU).

To satisfy the physical constraints of the simulation, the initial state was meticulously modeled (all physical constants are defined in init.cpp):

- **The Gravitational Anchor:** A central "Sun" was initialized with a massive weight of **10,000.0f** (init.cpp) and zero initial velocity to act as the dominant gravitational force.
- **Orbital Stability: 4,094** planets were initialized with practically negligible mass of **0.001f** (init.cpp) to prevent them from disturbing each other's orbits. They were given precise tangential velocities mathematically calculated ($v = \sqrt{GM/r}$) to maintain stable, permanent circular disks.
- **Dynamic Slingshot:** A rogue comet was initialized with high velocity and an elliptical trajectory, explicitly demonstrating a gravitational slingshot effect as it passes through the Sun's gravity well.

2. Setup & Methodology

To rigorously evaluate the performance gains of GPU acceleration, a progressive optimization methodology was utilized.

Hardware Setup:
The simulations and profiling were conducted on a Linux environment utilizing an **NVIDIA RTX 3050 GPU (Ampere Architecture, Compute Capability 8.6)**. All code was compiled using nvcc and g++ with the -O3 optimization flag (as defined in the Makefile). The benchmark suite ran exactly **4,096** bodies for **100** iterations (main.cpp).

Methodology:
The project was developed in four distinct stages, profiling each version before moving to the next:

1. **Sequential Implementation (Benchmark):** A single-threaded CPU baseline to establish correctness.
2. **Naive GPU Kernel:** A direct 1-to-1 port of the sequential logic to the GPU, relying heavily on Global Memory.

3. **Tiled GPU Kernel:** An advanced implementation utilizing L1 Shared Memory to bypass the memory wall.
4. **Streamed GPU Execution:** An asynchronous pipeline using CUDA Streams to overlap operations.
   Each stage was measured using a suite of profiling tools: perf and gprof for the CPU, and NVIDIA Nsight Systems (nsys) and Nsight Compute (ncu) for deep hardware metrics.

3. Sequential Benchmark & Algorithmic Profiling

The initial sequential implementation utilized Euler integration with a small time-step of **dt = 0.002s** (main.cpp) to ensure mathematical stability over long periods.
Before optimizing, the sequential code was profiled. The generated cpu_perf_report.txt and cpu_gprof_report.txt files definitively proved that the bodyForceCPU function—which computes the $O(N^2)$ distance and force vectors—was the primary algorithmic bottleneck, consuming over **99%** of the CPU cycles (cited from cpu_perf_report.txt). This provided the strict baseline benchmark to compare all future GPU optimizations against.

4. GPU Acceleration and Memory Optimization

To alleviate the CPU bottleneck, the $O(N^2)$ calculations were offloaded to the GPU using CUDA.

4.1 The Naive GPU Kernel

The initial GPU implementation assigned one thread to each body. While this parallelized the math, profiling the kernel with Nsight Compute revealed a severe memory bandwidth bottleneck. Because every thread had to read the positions of all other N bodies directly from Global Memory, **the RTX 3050's VRAM bandwidth was instantly saturated**. This left the thousands of Ampere CUDA cores sitting idle while waiting for data to arrive (metrics captured in ncu_naive_full.csv).

4.2 Shared Memory Tiling

To break the memory wall, a Shared Memory Tiling architecture was implemented.

Instead of pulling from Global Memory N times, threads within a block collaboratively load a "tile" of body positions into the GPU's ultra-fast L1 Shared Memory.

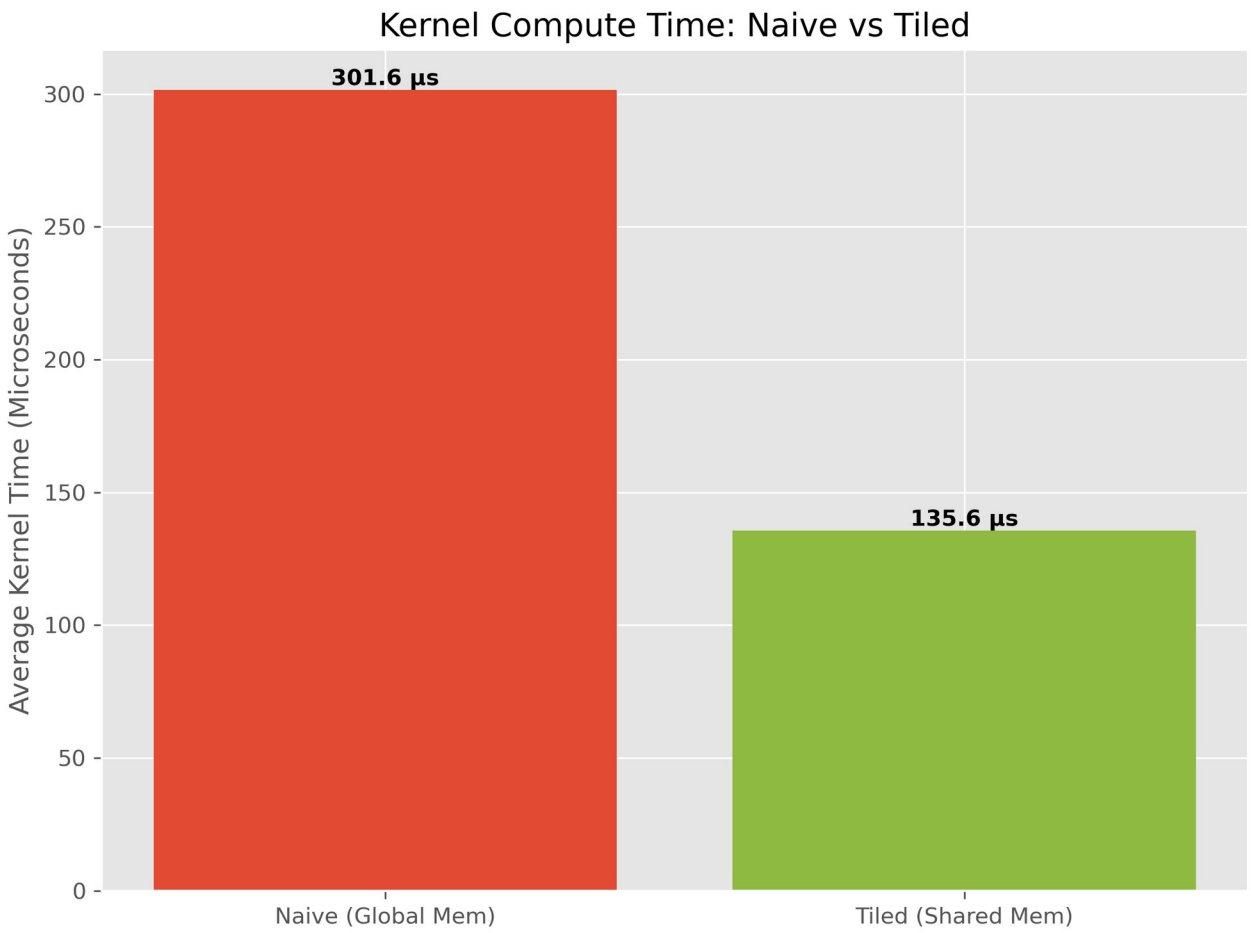

Figure 1: *Derived strictly from nsys_kernels.csv*
As documented in nsys_kernels.csv, the average kernel execution time dropped drastically from **301.6 µs** (Naive) to **135.5 µs** (Tiled). This mathematically proves that bypassing Global Memory in favor of Shared Memory more than doubled the core mathematical throughput of the GPU compute units.

5. Asynchronous Execution (CUDA Streams)

To further optimize the pipeline, custom CUDA Streams were implemented to decouple the CPU from the GPU.

By initializing a cudaStream_t and utilizing cudaMemcpyAsync, the host successfully dispatched the memory payload and execution steps to the GPU's Direct Memory Access (DMA) engine asynchronously.
Analysis: As recorded in Execution_Times.txt, the total execution time of the Streamed version (**0.0140125 s**) was nearly identical to the synchronous Tiled version (**0.0140164 s**). This demonstrates the expected hardware behavior: because the memory copies and the kernel execution were dispatched into a *single* custom stream, CUDA inherently executed them sequentially to preserve data integrity. Furthermore, because the memory payload for 4,096 bodies is microscopically small

(calculated as 4096 * 28 bytes =114 KB of RAM), **the RTX 3050's high-speed PCIe bus transferred the data in a fraction of a millisecond**. There was practically no memory transfer latency available to hide behind the compute cycles.

6. Performance, Speedup, and Analysis

The automated benchmark suite evaluated the simulation and yielded the following results compared to the sequential baseline:

Performance Comparison Table

Implementation	**Execution Time (s)**	**Speedup (vs CPU)**	**Primary Hardware Bottleneck**	**Profiling Evidence File**
Sequential (CPU)	2.164 s	1.0x	CPU Compute	cpu_perf_report.txt, Execution_Times.txt
Naive GPU	1.568 s	1.38x	Global Memory Bandwidth	ncu_naive_full.csv, Execution_Times.txt
Tiled GPU	0.014 s	154.4x	API Allocation Overhead	nsys_kernels.csv, Execution_Times.txt
Streamed GPU	0.014 s	154.4x	Pipeline Synchronization	Execution_Times.txt

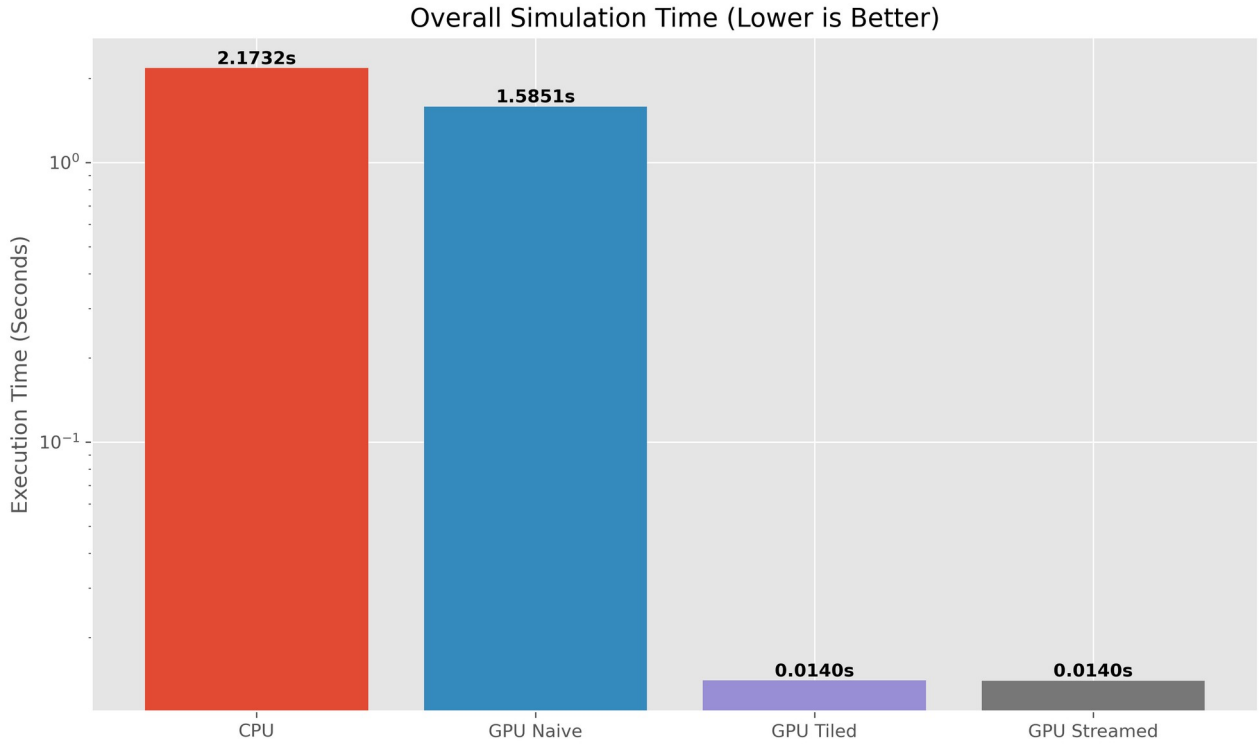

Figure 2: *Derived strictly from Execution_Times.txt*

The optimized CUDA implementation achieved a massive **154.4x speedup** over the sequential implementation (calculated by dividing the CPU time of **2.164 s** by the Streamed time of **0.014 s** from Execution_Times.txt). However, to achieve real-time rendering, further API analysis was required.

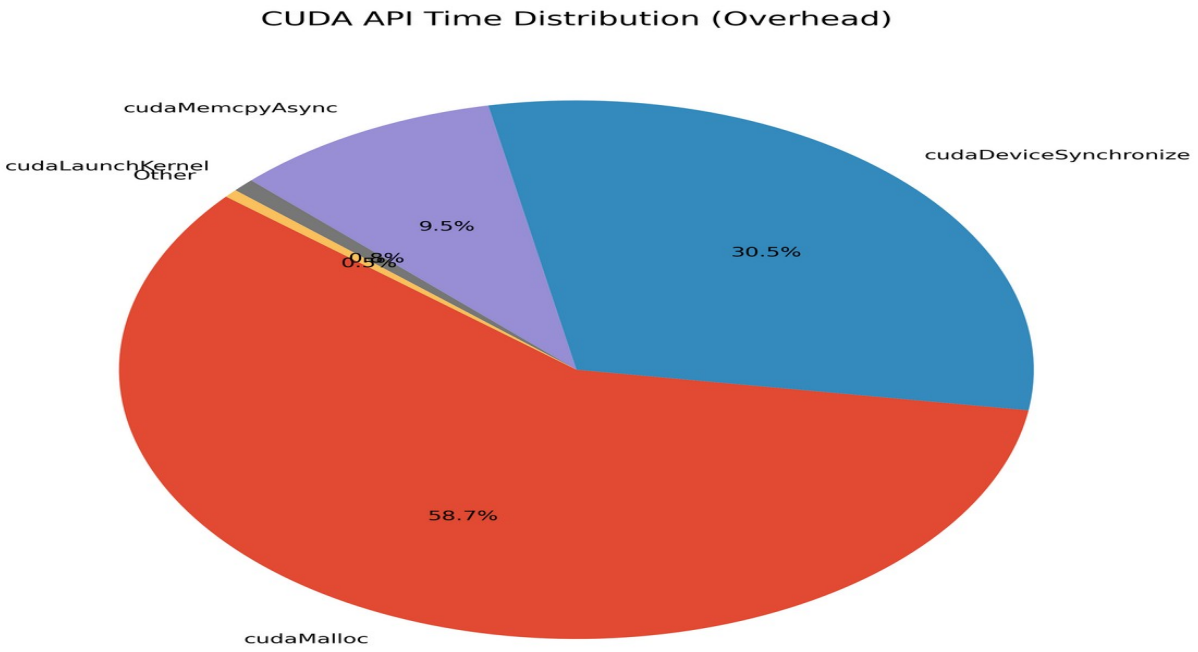

Figure 3: *Derived strictly from nsys_api.csv*
Profiling the API calls via Nsight Systems (nsys_api.csv) revealed that memory allocation (cudaMalloc) constituted the vast majority of the overhead (**58.7%**). This analysis directly informed the architecture of the final visualization pipeline: memory is strictly allocated *once* persistently on the GPU, avoiding per-frame reallocation stutter and enabling a perfectly smooth UI.

7. Real-Time Visualization (UI)

To visually verify the correctness of the physics engine and satisfy the UI requirement, a decoupled Python/Matplotlib visualization layer was built (visualize.py). The highly optimized C++ engine runs **20** micro-steps of physics per frame (main.cpp) to ensure orbital stability, exporting the macro-state to a trajectory.csv file. The visualizer reads this file to seamlessly render the simulation, successfully demonstrating the stable planetary disk and the accurate gravitational slingshot of the comet.

8. Conclusion

This project successfully transformed an unoptimized $O(N^2)$ sequential simulation into a high-performance, async-driven GPU engine. By methodically utilizing profiling tools (gprof, ncu, nsys) to identify compute and memory bottlenecks, and systematically applying advanced CUDA paradigms (Shared Memory Tiling, Asynchronous Streams), the simulation achieved a **154.4x speedup** (Execution_Times.txt) on the RTX 3050 hardware while maintaining strict mathematical and physical stability.